Urdaneta City University
College of Information and Technology Education

Midterm Laboratory 2

Converting a Portfolio Section into a Dynamic CRUD Feature
Laravel, Inertia.js and React

Name	Ahrone B. Fabro
Course	IT ELEC01 – Advanced Web Development (C287-C288)
Section	BSIT 3-4
Instructor	Giobet T. Aban
Date	November 2, 2026

1. Section Chosen and Justification

Which section did you choose, and why?

I chose the **Projects** section. It is the part of my portfolio that changes the most, because I finish new work every semester and my older projects may need new links or descriptions. It also has the most varied data: text, a date, a link, an image and a set of categories, so it gives me a good reason to use many kinds of validation.

Managing projects through a database makes more sense than hard-coding them. With hard-coded cards, I would have to edit a React file and rebuild the site just to fix a typo. Now I can add, edit or delete a project through a form, and the public page updates by itself. The card design is written once in `ProjectCard.jsx` and is reused for every record.

What are your fields?

The table has **nine meaningful fields**, not counting `id` and the timestamps.

Field	Data type	Why it is needed
`title`	string(60), unique	The heading of the card. It is unique so two cards cannot have the same name.
`categories`	json	A project can belong to more than one category, such as mobile and design. The filter buttons on the public page use this field.
`description`	text	A short summary that tells visitors what the project does.

`tech_stack`	string(120)	The tools I used, separated by commas. Each one is shown as a small tag on the card.
`project_url`	string, nullable	A link to the live site, Figma file or repository. It is optional because not every project has a public link.
`image`	string	The path of the uploaded cover image. The file itself is kept in `storage/app/public`, and only its path goes in the database.
`image_alt`	string(200)	A description of the image for visitors who use a screen reader.
`date_completed`	date	Shows when I finished the project.
`display_order`	unsigned small integer	Lets me choose which projects appear first. A lower number shows first.

What rules must your data follow?

1. A title is required, must be 3 to 60 characters long, and cannot be the same as another project's title.
2. A project must have at least one category, and each category must be web, mobile, design or school.
3. A description is required and must be 20 to 300 characters long.
4. The tools used are required, with at most 120 characters.
5. A project link is optional, but if there is one, it must be a valid http or https URL.
6. A cover image is required when creating a project and optional when editing. It must be a JPG, PNG or WEBP image of 2 MB or less.
7. The image description is required and must be 10 to 200 characters long.
8. The completion date is required and cannot be in the future.
9. The display order is required and must be a whole number from 1 to 99.

2. Screenshots of All Pages

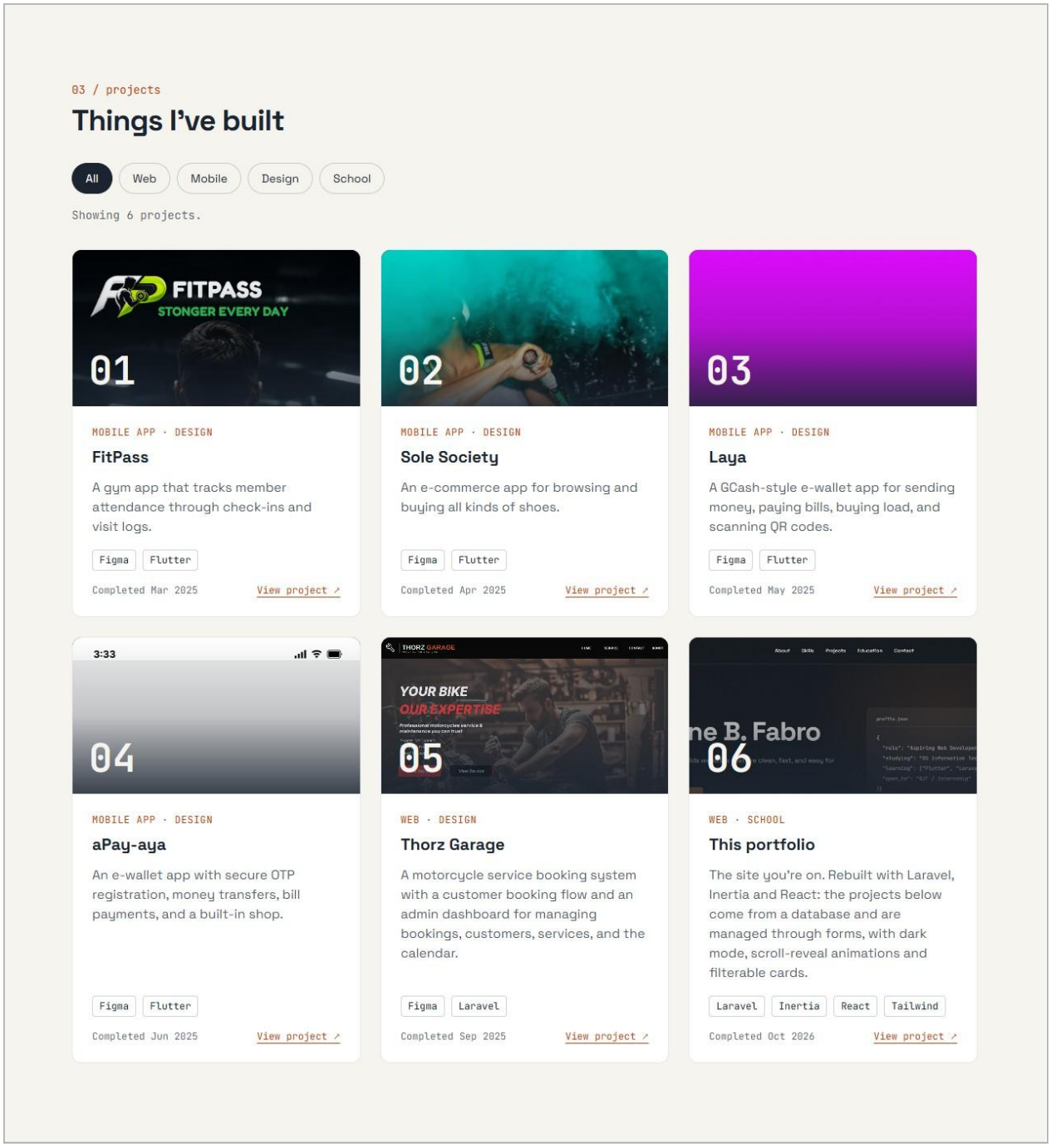


Figure A. Public portfolio page showing the dynamic Projects section, loaded from the database.

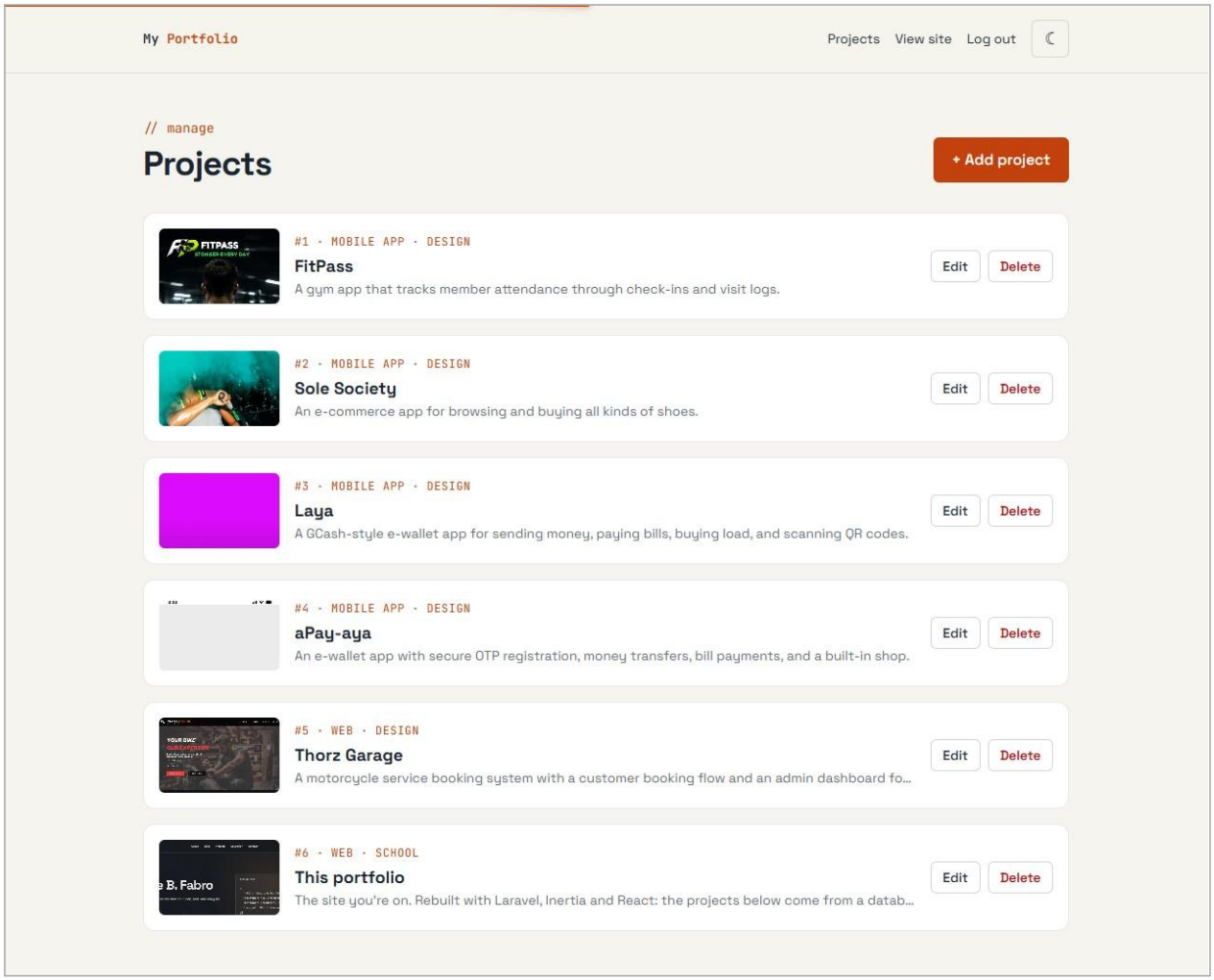


Figure B. Index page: the list of all project records with Edit and Delete buttons.

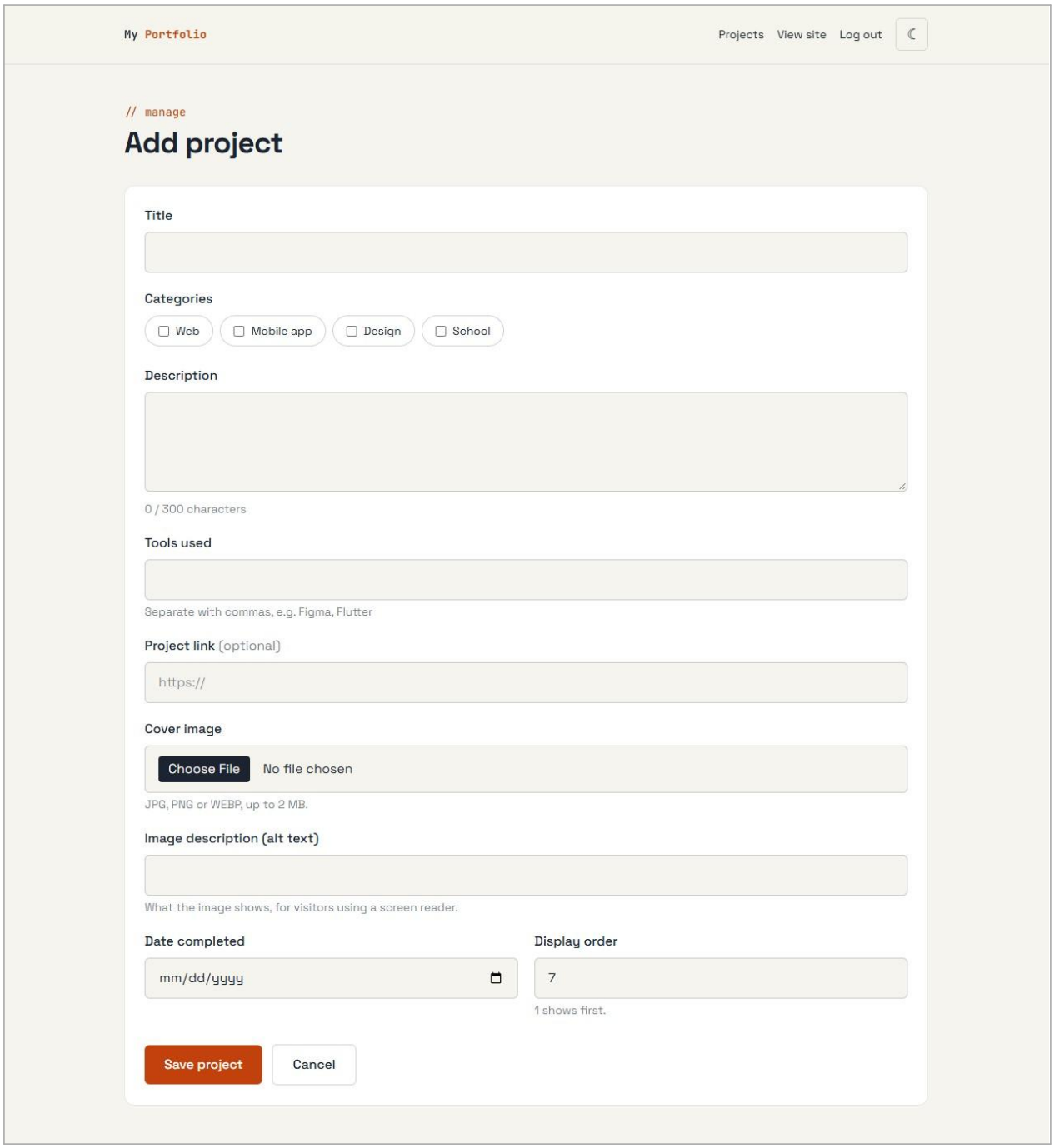


Figure C. Create page: the empty form for adding a project.

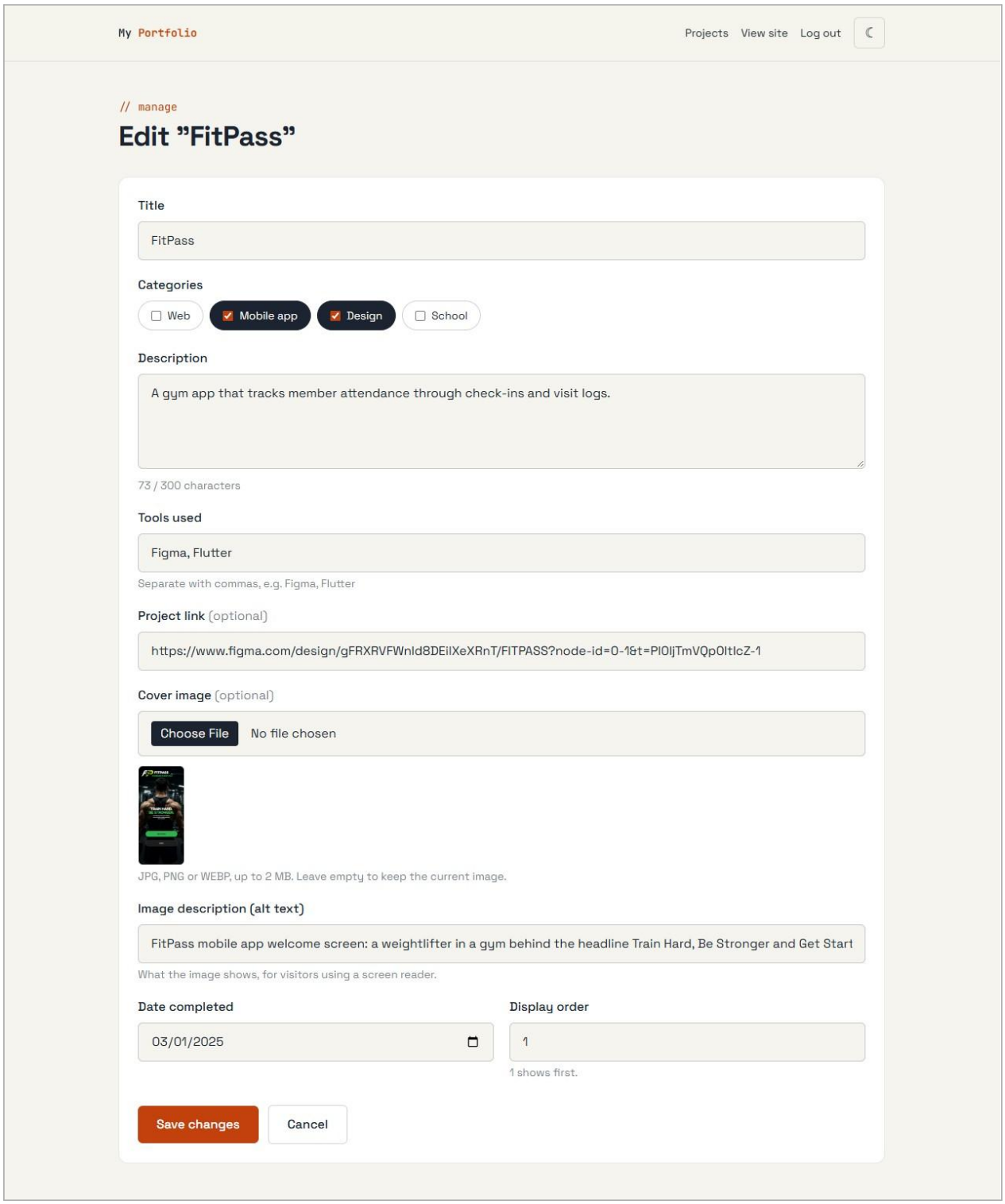


Figure D. Edit page: the same form, filled with the current values of a project.

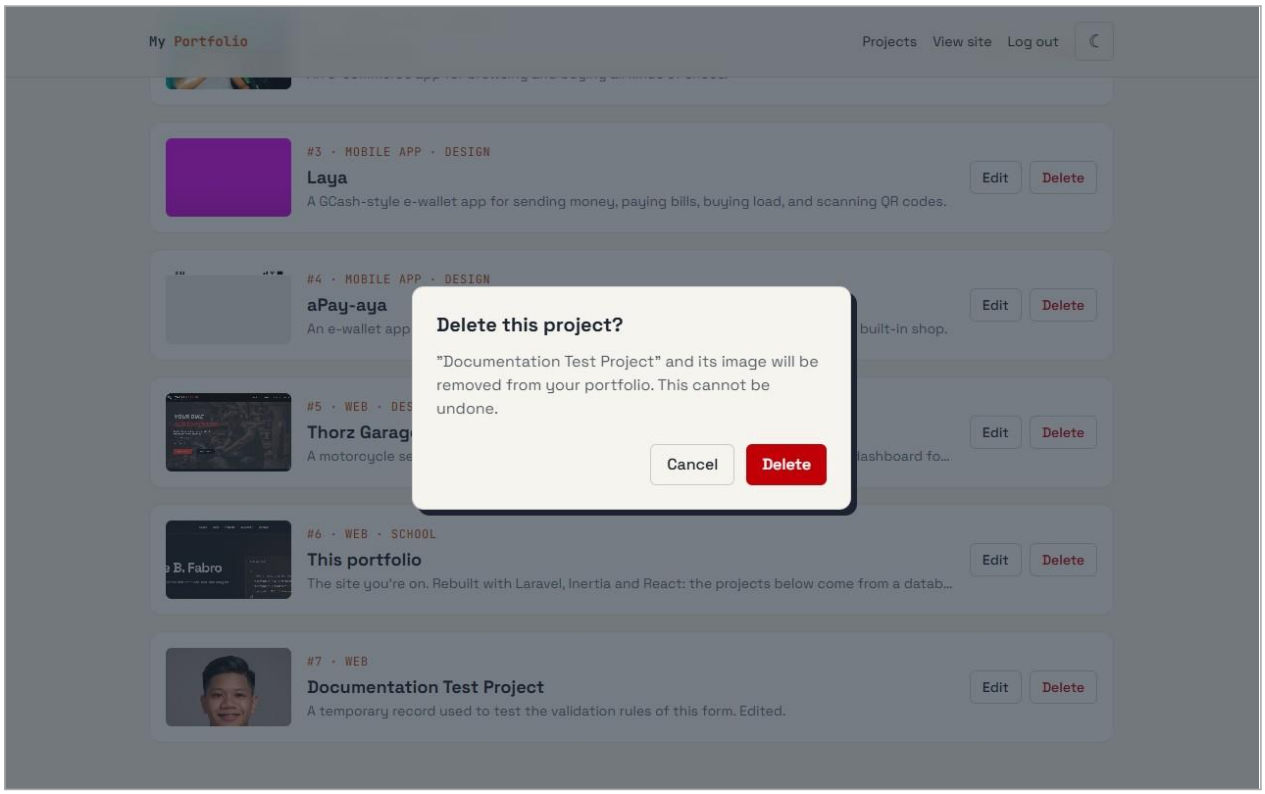


Figure E. Delete confirmation: a dialog asks before the record is removed.

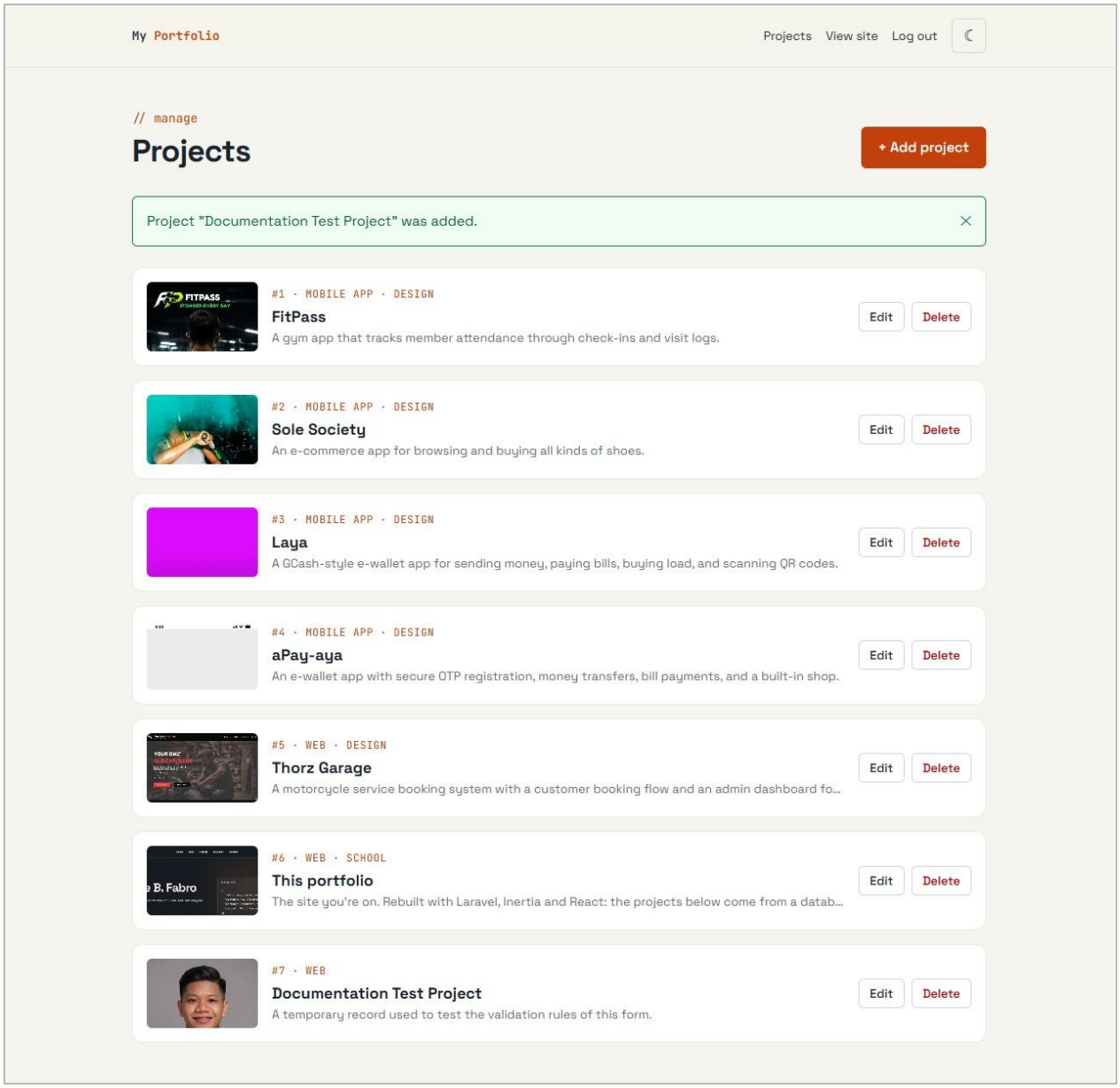


Figure F. Success message shown after creating a project. The last record is the temporary test project, which was deleted afterwards.

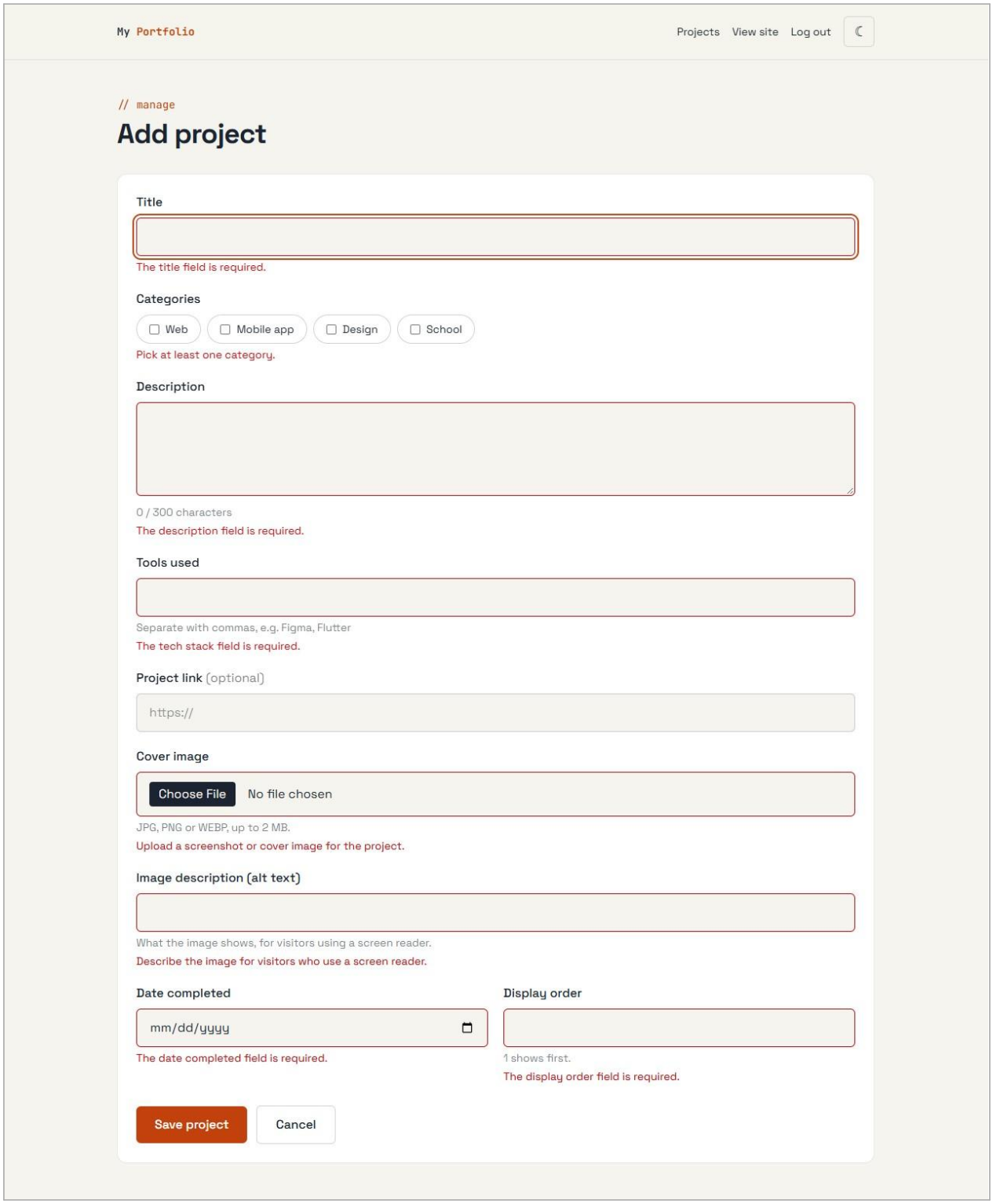


Figure G. Validation errors displayed under each field after submitting an empty form.

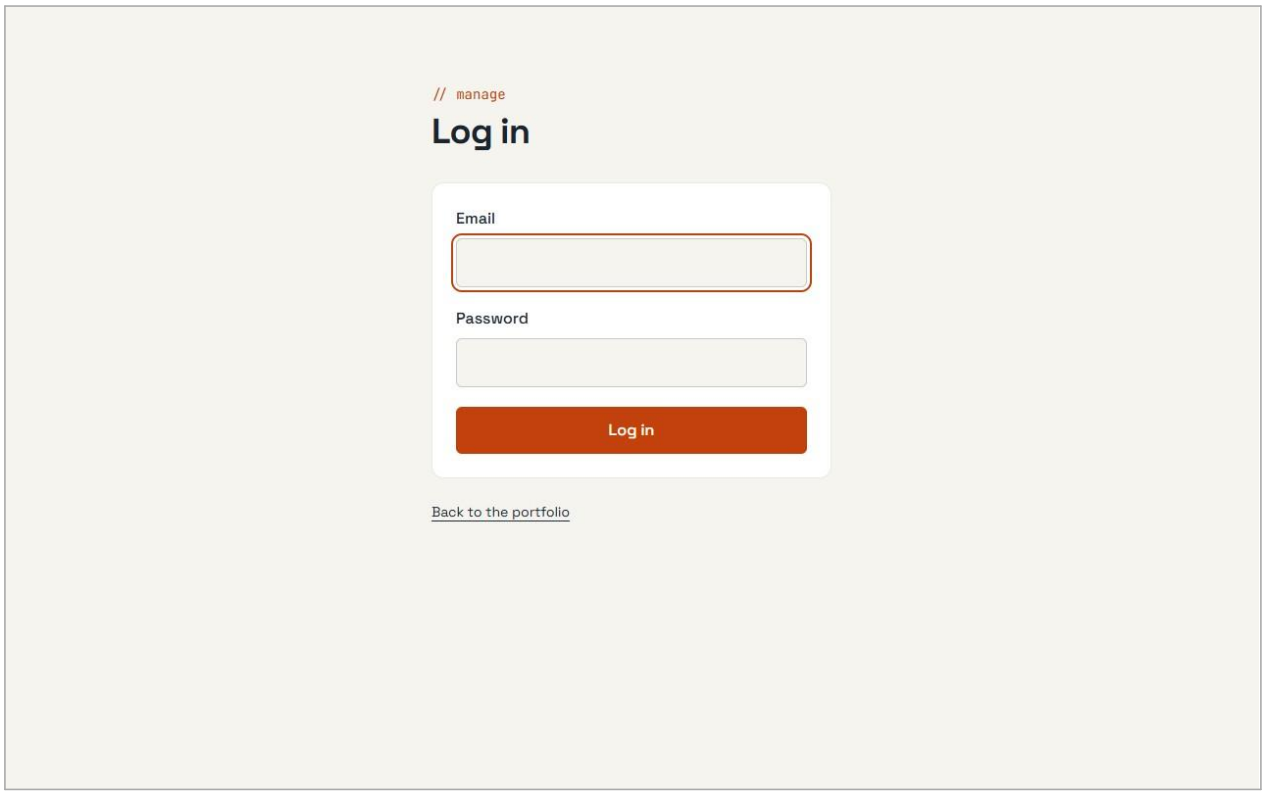


Figure H. Login page: visitors who are not logged in are sent here when they open any project management page.

3. Folder Structure

These are the files created or modified for the Projects CRUD and its login.

```
app/
  Http/
    Controllers/
      LoginController.php         log in and log out
      PortfolioController.php     public portfolio page
      ProjectController.php       CRUD and validation rules
    Middleware/
      HandleInertiaRequests.php   shares the success message with React
  Models/
    Project.php bootstrap/   app.php                          where
logged-in users are redirected database/   migrations/
    2026_10_21_121541_create_projects_table.php
seeders/
    DatabaseSeeder.php
    ProjectSeeder.php             loads my 6 existing
projects routes/   web.php   console.php
admin:create command resources/js/
  Pages/
    Login.jsx                     login form
    Portfolio.jsx                 public portfolio page
    Projects/
      Index.jsx
      Create.jsx
      Edit.jsx
  Components/
    ProjectForm.jsx               form shared by Create and Edit
    FormField.jsx                 label, input, hint and error message
    ConfirmDialog.jsx             delete confirmation
    FlashMessage.jsx              success message
    ProjectsSection.jsx           public section with filter buttons
    ProjectCard.jsx               one project on the public page
    Navbar.jsx                    top bar of the public page
    Footer.jsx                    bottom of the public page
    SectionTitle.jsx              label and heading of each section
    ThemeToggle.jsx               light and dark mode button
    ScrollProgress.jsx            scroll bar at the top of the public page
    ContactForm.jsx               contact form with autosaved draft
  Layouts/
    AdminLayout.jsx               frame of the management pages
tests/Feature/
  LoginTest.php                   automated login and access tests
  ProjectCrudTest.php             automated CRUD and validation tests
```

4. Source Code

a. Migration

`database/migrations/2026_10_21_121541_create_projects_table.php`

This file creates the `projects` table and its nine fields. Each column has the type that fits its data, like `json` for categories and `date` for the completion date. The database also enforces the unique title, as a second layer behind the controller validation.

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint; use
Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**      * Run the
migrations.
     */
    public function up(): void
    {
        Schema::create('projects', function (Blueprint $table) {
            $table->id();
            $table->string('title', 60)->unique();      // card heading; unique so two cards can't share a name
            $table->json('categories');                  // e.g. ["mobile","design"], drives the filter buttons
            $table->text('description');                 // short summary shown on the card
            $table->string('tech_stack', 120);           // comma-separated tools, e.g. "Figma, Flutter"
            $table->string('project_url')->nullable();   // live site / Figma / repo link (optional)
            $table->string('image');                     // path inside storage/app/public
            $table->string('image_alt', 200);            // alt text so the image is described to screen readers
            $table->date('date_completed');
            $table->unsignedSmallInteger('display_order')->default(1); // lower number shows first
            $table->timestamps();
        });
    }

    /**      * Reverse the
migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('projects');
    }
};
```

b. Model

app/Models/Project.php

The model connects the `projects` table to my PHP code. `$fillable` lists the only columns a form may fill, which protects fields like `id`. The casts turn the categories JSON into an array, and `image_url` adds the public image address that React needs.

```php
<?php namespace

App\Models;

use Illuminate\Database\Eloquent\Casts\Attribute;
use Illuminate\Database\Eloquent\Model;

class Project extends Model
{
    // The categories a project can be filed under (also the filter buttons on the portfolio)
public const CATEGORIES = ['web', 'mobile', 'design', 'school'];

    // Only these columns can be filled from a form, so a request can't sneak in "id" or timestamps
protected $fillable = [
        'title',
        'categories',
        'description',
        'tech_stack',
        'project_url',
        'image',
        'image_alt',
        'date_completed',
        'display_order',
    ];

    // Sent to React along with the columns
protected $appends = ['image_url'];

    protected function casts(): array
    {
        return [
            'categories' => 'array',        // JSON column <-> PHP array
            'date_completed' => 'date:Y-m-d',
            'display_order' => 'integer',
        ];
    }

    // Public URL of the uploaded image (served through the storage:link folder)
protected function imageUrl(): Attribute
    {
        return Attribute::get(fn () => $this->image ? '/storage/'.$this->image : null);
    }
}
```

c. Controllers (with validation rules and custom messages)

app/Http/Controllers/ProjectController.php

This resource controller holds the six CRUD actions. `rules()` is shared by `store` and `update`: when a project is passed in, the unique title check ignores that record and the image becomes optional. `messages()` holds my custom error messages. Old image files are removed only after the database change succeeds, so a failed save cannot leave a project with a broken image.

```
<?php namespace

App\Http\Controllers;

use App\Models\Project; use
Illuminate\Http\Request; use
Illuminate\Support\Facades\Storage; use
Illuminate\Validation\Rule; use
Inertia\Inertia;

class ProjectController extends Controller
{
    /**      * List every project for the
management page.
     */
    public function index()
    {
        return Inertia::render('Projects/Index', [
            'projects' => Project::orderBy('display_order')->orderBy('title')->get(),
        ]);
    }

    /**      * Show the
empty form.
     */
    public function create()
    {
        return Inertia::render('Projects/Create', [
            'categories' => Project::CATEGORIES,
            'nextOrder' => (Project::max('display_order') ?? 0) + 1,
        ]);
    }

    /**      * Validate and save a new
project.
     */
    public function store(Request $request)
    {
        $data = $request->validate($this->rules(), $this->messages());

$data['image'] = $request->file('image')->store('projects', 'public');

Project::create($data);

        return redirect()->route('projects.index')
            ->with('success', "Project \"{$data['title']}\" was added.");
    }

    /**      * Show the form filled with the project's current
values.
     */
    public function edit(Project $project)
    {
        return Inertia::render('Projects/Edit', [
            'project' => $project,
            'categories' => Project::CATEGORIES,
        ]);
    }

    /**      * Validate and save changes to a
project.
     */
    public function update(Request $request, Project $project)
    {
        $data = $request->validate($this->rules($project), $this->messages());

        $oldImage = $project->image;
```

```
        // Keep the old image unless a new one was uploaded
if ($request->hasFile('image')) {
            $data['image'] = $request->file('image')->store('projects', 'public');
        } else {
            unset($data['image']);
}

        $project->update($data);

        // Only remove the old file once the new one is saved, so a failure can't leave a broken image
if ($project->image !== $oldImage) {
            Storage::disk('public')->delete($oldImage);
        }

        return redirect()->route('projects.index')
            ->with('success', "Project \"{$project->title}\" was updated.");
    }

    /**      * Delete a project and its
image file.
     */
    public function destroy(Project $project)
    {
        $project->delete();
        Storage::disk('public')->delete($project->image);

        return redirect()->route('projects.index')
            ->with('success', "Project \"{$project->title}\" was deleted.");
    }

    /**
     * Rules shared by store and update. On update, $project is passed so the
* unique title check ignores this record and the image becomes optional.
     */
    private function rules(?Project $project = null): array
    {
        return [
            'title' => [
                'required', 'string', 'min:3', 'max:60',
                Rule::unique('projects', 'title')->ignore($project),
            ],
            'categories' => ['required', 'array', 'min:1'],
            'categories.*' => ['distinct', Rule::in(Project::CATEGORIES)],
            'description' => ['required', 'string', 'min:20', 'max:300'],
            'tech_stack' => ['required', 'string', 'max:120'],
            'project_url' => ['nullable', 'url:http,https', 'max:255'],
            'image' => [$project ? 'nullable' : 'required', 'image', 'mimes:jpg,jpeg,png,webp', 'max:2048'],
            'image_alt' => ['required', 'string', 'min:10', 'max:200'],
            'date_completed' => ['required', 'date', 'before_or_equal:today'],
            'display_order' => ['required', 'integer', 'between:1,99'],
        ];
    }

    /**      * Custom error messages, written for the person filling in
the form.
     */
    private function messages(): array
    {
        return [
            'title.unique' => 'You already have a project with this title.',
            'categories.required' => 'Pick at least one category.',
            'categories.min' => 'Pick at least one category.',
            'categories.*.in' => 'That category does not exist.',
            'description.min' => 'The description should be at least 20 characters so visitors know what the project
is.',
            'project_url.url' => 'The project link must be a full web address starting with http:// or https://.',
            'image.required' => 'Upload a screenshot or cover image for the project.',
            'image.image' => 'The file must be an image.',
            'image.mimes' => 'The image must be a JPG, PNG or WEBP file.',
            'image.max' => 'The image must be 2 MB or smaller.',
            'image_alt.required' => 'Describe the image for visitors who use a screen reader.',
            'date_completed.before_or_equal' => 'The completion date cannot be in the future.',
            'display_order.between' => 'The display order must be a number from 1 to 99.',
        ];
    }
}
```

app/Http/Controllers/PortfolioController.php

This controller feeds the public page. It reads the projects in their display order and passes them to the React `Portfolio` page through Inertia.

```
<?php namespace

App\Http\Controllers;

use App\Models\Project;
use Inertia\Inertia;

class PortfolioController extends Controller
{
    /**
     * The public portfolio page. Projects come from the database;
* every other section is still written directly in the React page.
     */
    public function index()
    {
        return Inertia::render('Portfolio', [
            'projects' => Project::orderBy('display_order')->orderBy('title')->get(),
        ]);
    }
}
```

app/Http/Controllers/LoginController.php

This controller shows the login form, checks the email and password, and logs the user out. The session is renewed at login and ended at logout.

```
<?php namespace

App\Http\Controllers;

use Illuminate\Http\Request; use
Illuminate\Support\Facades\Auth; use
Illuminate\Validation\ValidationException; use
Inertia\Inertia;

class LoginController extends Controller
{
    /**      * Show the
login form.
     */
    public function create()
    {
        return Inertia::render('Login');
}

    /**      * Log the owner in and continue to the page they were trying
to open.
     */
    public function store(Request $request)
    {
        $credentials = $request->validate([
            'email' => ['required', 'email'],
            'password' => ['required', 'string'],
        ]);

        if (! Auth::attempt($credentials)) {
            throw ValidationException::withMessages([
                'email' => 'That email and password do not match.',
            ]);
        }         $request->session()-

>regenerate();

        return redirect()->intended(route('projects.index'));
}

    /**      * Log out and go back to the public
portfolio.
     */
    public function destroy(Request $request)
    {
        Auth::logout();

        $request->session()->invalidate();
        $request->session()->regenerateToken();

        return redirect()->route('portfolio');
    }
}
```

d. Routes

routes/web.php

The first route is the public portfolio. The login routes are limited to 5 attempts a minute.

`Route::resource` creates the index, create, store, edit, update and destroy routes from one line, inside the `auth` group so they require a login. `show` is left out because each project is already shown in full on its card.

```php
<?php

use App\Http\Controllers\LoginController; use
App\Http\Controllers\PortfolioController; use
App\Http\Controllers\ProjectController; use
Illuminate\Support\Facades\Route;

// Public portfolio
Route::get('/', [PortfolioController::class, 'index'])->name('portfolio');

// Login. Attempts are limited to 5 a minute to slow down password guessing.
Route::middleware('guest')->group(function () {
    Route::get('/login', [LoginController::class, 'create'])->name('login');
    Route::post('/login', [LoginController::class, 'store'])->middleware('throttle:5,1');
});

Route::middleware('auth')->group(function () {
    Route::post('/logout', [LoginController::class, 'destroy'])->name('logout');

    // Project management: index, create, store, edit, update, destroy.
    // "show" is left out because each project is already shown in full on its card.
    Route::resource('projects', ProjectController::class)->except('show');
});
```

routes/console.php

There is no public sign-up page, so the `admin:create` command creates the login or changes its password.

```php
<?php

use App\Models\User; use
Illuminate\Foundation\Inspiring; use
Illuminate\Support\Facades\Artisan;

Artisan::command('inspire', function () {
    $this->comment(Inspiring::quote());
})->purpose('Display an inspiring quote');

Artisan::command('admin:create {email}', function (string $email) {
    $password = (string) $this->secret('Password (at least 8 characters)');

    if (strlen($password) < 8) {         $this->error('The password
must be at least 8 characters.');

        return 1;
}

    User::updateOrCreate(['email' => $email], ['name' => 'Admin', 'password' => $password]);
    $this->info("{$email} can now log in at /login.");
})->purpose('Create the login for the project pages, or change its password');
```

bootstrap/app.php

This registers the Inertia middleware and sends users who are already logged in from `/login` to the project list.

```php
<?php

use Illuminate\Foundation\Application; use
Illuminate\Foundation\Configuration\Exceptions; use
Illuminate\Foundation\Configuration\Middleware;

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
commands: __DIR__.'/../routes/console.php',
health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware) {
        $middleware->web(append: [
            \App\Http\Middleware\HandleInertiaRequests::class,
        ]);

        // Already logged in and opening /login: go to the management page
        $middleware->redirectUsersTo('/projects');
    })
    ->withExceptions(function (Exceptions $exceptions) {
        //
    })->create();
```

e. React Pages

resources/js/Pages/Projects/Index.jsx

This page lists every project with its Edit and Delete buttons. Clicking Delete only stores which project was chosen and opens the confirmation dialog. The record is deleted only when the user confirms.

```
import { Link, router } from '@inertiajs/react'; import {
useState } from 'react'; import ConfirmDialog from
'../../Components/ConfirmDialog'; import { categoryLabels }
from '../../Components/ProjectCard'; import AdminLayout from
'../../Layouts/AdminLayout';

const btn = 'rounded-md border border-ink/20 px-3 py-1.5 text-sm font-medium hover:border-ink dark:border-chalk/20
dark:hover:border-chalk';

// List of every project, with Edit and Delete for each. export default function Index({
projects }) {   const [toDelete, setToDelete] = useState(null); // the project waiting
for confirmation   const [deleting, setDeleting] = useState(false);

  function confirmDelete() {
router.delete(`/projects/${toDelete.id}`, {
preserveScroll: true,       onStart: () => setDeleting(true),
onFinish: () => { setDeleting(false); setToDelete(null); },
    });
  }

  return (
    <AdminLayout
title="Projects"
actions={
        <Link href="/projects/create" className="rounded-md bg-accent px-5 py-3 font-medium text-white transition
hover:bg-ink dark:bg-ember dark:text-night dark:hover:bg-chalk">
          + Add project
        </Link>
      }
    >
      {projects.length === 0 ? (
        <p className="rounded-xl border border-dashed border-ink/20 p-10 text-center text-ink/70 dark:border-chalk/20
dark:text-chalk/70">
          No projects yet. <Link href="/projects/create" className="text-accent underline dark:text-ember">Add your
first one.</Link>
        </p>
      ) : (
        <ul className="space-y-4">
          {projects.map((project) => (
            <li key={project.id} className="flex flex-col gap-4 rounded-xl border border-ink/10 bg-white p-4 sm:flex-
row sm:items-center dark:border-chalk/10 dark:bg-white/5">
              <img src={project.image_url} alt="" className="h-20 w-full rounded-md object-cover object-top sm:w-32"
/>
              <div className="min-w-0 flex-1">
                <p className="font-mono text-xs uppercase tracking-wider text-accent dark:text-ember">
                  #{project.display_order} · {project.categories.map((c) => categoryLabels[c] ?? c).join(' · ')}
                </p>
                <h2 className="mt-1 text-lg font-bold">{project.title}</h2>
                <p className="truncate text-sm text-ink/70 dark:text-chalk/70">{project.description}</p>
              </div>
              <div className="flex gap-2">
                <Link href={`/projects/${project.id}/edit`} className={btn}>Edit<span className="sr-only">
{project.title}</span></Link>
                <button type="button" onClick={() => setToDelete(project)} className={`${btn} text-red-700
hover:border-red-700 dark:text-red-400 dark:hover:border-red-400`}>
                  Delete<span className="sr-only"> {project.title}</span>
                </button>
              </div>
            </li>
          ))}
        </ul>
      )}

      <ConfirmDialog
        open={toDelete !== null}         title="Delete this project?"         message={toDelete ?
`"${toDelete.title}" and its image will be removed from your portfolio. This cannot be undone.` : ''}
processing={deleting}
```

```
        onConfirm={confirmDelete}
onCancel={() => setToDelete(null)}
      />
    </AdminLayout>
  );
}
```

resources/js/Pages/Projects/Create.jsx

This page starts `useForm` with empty values and passes it to the shared form. It sends the data as multipart form data so the image file is included.

```
import { useForm } from '@inertiajs/react'; import
ProjectForm from '../../Components/ProjectForm'; import
AdminLayout from '../../Layouts/AdminLayout';

export default function Create({ categories, nextOrder }) {
const form = useForm({     title: '',     categories: [],
description: '',     tech_stack: '',     project_url: '',
image: null,     image_alt: '',     date_completed: '',
display_order: nextOrder,   });

  // forceFormData: send as multipart so the image file is included
  const submit = () => form.post('/projects', { forceFormData: true, preserveScroll: true });

  return (
    <AdminLayout title="Add project">
      <ProjectForm form={form} categories={categories} onSubmit={submit} submitLabel="Save project" />
    </AdminLayout>
  );
}
```

resources/js/Pages/Projects/Edit.jsx

This page starts `useForm` with the current values of the project. Browsers cannot send files with a real PUT request, so it sends a POST with `_method: put`, and Laravel treats it as a PUT that reaches `update`.

```
import { useForm } from '@inertiajs/react'; import
ProjectForm from '../../Components/ProjectForm'; import
AdminLayout from '../../Layouts/AdminLayout';

export default function Edit({ project, categories }) {
const form = useForm({     title: project.title,
categories: project.categories,     description:
project.description,     tech_stack:
project.tech_stack,     project_url:
project.project_url ?? '',     image: null, // empty =
keep the current image     image_alt:
project.image_alt,     date_completed:
project.date_completed,     display_order:
project.display_order,   });

  // Browsers can't send files with a real PUT request, so this POSTs with _method=put
  // and Laravel treats it as PUT (it still reaches ProjectController@update).   const
submit = () => {     form.transform((data) => ({ ...data, _method: 'put' }));
form.post(`/projects/${project.id}`, { forceFormData: true, preserveScroll: true });
};

  return (
    <AdminLayout title={`Edit "${project.title}"`}>
      <ProjectForm form={form} categories={categories} onSubmit={submit} submitLabel="Save changes"
currentImageUrl={project.image_url} />
    </AdminLayout>
  );
}
```

resources/js/Pages/Login.jsx

This page shows the email and password fields and the error message from the server.

```
import { Head, Link, useForm } from '@inertiajs/react'; import
FormField, { inputClass } from '../Components/FormField';

// Login for the project management pages. There is no sign-up: the account is made with "php artisan admin:create".
export default function Login() {
  const { data, setData, post, errors, processing } = useForm({ email: '', password: '' });

  function handleSubmit(e) {
    e.preventDefault();     post('/login', { onError: () =>
setData('password', '') });   }

  const a11y = (field) => ({
id: field,
    'aria-invalid': errors[field] ? 'true' : 'false',
    'aria-describedby': `${field}-error`,
  });

  return (
    <>
      <Head title="Log in" />
      <main id="main" className="mx-auto max-w-md px-5 py-20">
        <p className="font-mono text-sm text-accent dark:text-ember">// manage</p>
        <h1 className="mt-2 mb-8 text-3xl font-bold tracking-tight sm:text-4xl">Log in</h1>

        <form onSubmit={handleSubmit} noValidate className="space-y-5 rounded-xl border border-ink/10 bg-white p-6
dark:border-chalk/10 dark:bg-white/5">
          <FormField id="email" label="Email" error={errors.email}>
            <input {...a11y('email')} type="email" autoComplete="username" autoFocus value={data.email} onChange={(e)
=> setData('email', e.target.value)} className={inputClass} />
          </FormField>
          <FormField id="password" label="Password" error={errors.password}>
            <input {...a11y('password')} type="password" autoComplete="current-password" value={data.password}
onChange={(e) => setData('password', e.target.value)} className={inputClass} />
          </FormField>
          <button type="submit" disabled={processing} className="w-full rounded-md bg-accent px-6 py-3 font-medium
text-white transition hover:bg-ink disabled:opacity-60 dark:bg-ember dark:text-night dark:hover:bg-chalk">
            {processing ? 'Logging in…' : 'Log in'}
          </button>
        </form>

        <p className="mt-6 text-sm"><Link href="/" className="underline underline-offset-4 hover:text-accent
dark:hover:text-ember">Back to the portfolio</Link></p>
      </main>
    </>
  );
}
```

resources/js/Pages/Portfolio.jsx

This is the public portfolio page. The Projects section gets its data from the `projects` prop sent by the controller, while the other sections are still written directly in this file.

```
import { Head } from '@inertiajs/react'; import { useEffect,
useRef, useState } from 'react'; import ContactForm from
'../Components/ContactForm'; import Footer from
'../Components/Footer'; import Navbar from
'../Components/Navbar'; import ProjectsSection from
'../Components/ProjectsSection'; import ScrollProgress from
'../Components/ScrollProgress'; import SectionTitle from
'../Components/SectionTitle'; import useScrollReveal from
'../hooks/useScrollReveal';

const navLinks = [
  { id: 'about', label: 'About' },
  { id: 'skills', label: 'Skills' },
  { id: 'projects', label: 'Projects' },
  { id: 'education', label: 'Education' },
  { id: 'contact', label: 'Contact' },
];

const skillGroups = [
  { title: 'Front-end', style: 'solid', items: ['HTML5', 'Tailwind CSS', 'JavaScript', 'CSS3'] },
  { title: 'App & back-end', style: 'solid', items: ['Flutter', 'Dart', 'Firebase', 'Laravel', 'PHP', 'MySQL'] },
  { title: 'Tools', style: 'outline', items: ['Git & GitHub', 'VS Code', 'Figma', 'Chrome DevTools'] },
  { title: 'Soft skills', style: 'dashed', items: ['Communication', 'Teamwork', 'Problem solving', 'Time management']
},
];

const skillChip = {   solid: 'rounded-md bg-ink px-3 py-1 text-paper dark:bg-
chalk dark:text-night',   outline: 'rounded-md border border-ink/20 px-3 py-1
dark:border-chalk/20',
  dashed: 'rounded-md border border-dashed border-accent px-3 py-1 dark:border-ember',
};

const education = [
  { years: '2024 – Present', title: 'Bachelor of Science in Information Technology', school: 'Urdaneta City
University', current: true },
  { years: '2017 – 2023', title: 'Junior & Senior High School', school: 'Lananpin National High School' },
  { years: '2011 – 2017', title: 'Elementary', school: 'Lananpin Elementary School' },
];

const profileCode = `{
  "role": "Aspiring Web Developer",
  "studying": "BS Information Technology",
  "learning": ["Flutter", "Laravel", "Tailwind"],
  "open_to": "OJT / Internship"
}`;

const marqueeText = 'HTML5 ✦ Tailwind CSS ✦ CSS3 ✦ JavaScript ✦ Flutter ✦ Laravel ✦ Accessibility ✦ Responsive Design
✦ UI / UX ✦';

const reduceMotion = () => window.matchMedia('(prefers-reduced-motion: reduce)').matches;
const stagger = (i) => ({ '--d': `${i * 120}ms` });

// Profile card that types itself out
function TypedCode() {
  const [length, setLength] = useState(() => (reduceMotion() ? profileCode.length : 0));

  useEffect(() => {     if (reduceMotion()) return;     const introDelay =
document.documentElement.classList.contains('no-intro') ? 0 : 1300; // matches --intro in CSS     let pos = 0;
let timer;

    function typeNext() {       pos++;       setLength(pos);       if (pos < profileCode.length) {
timer = setTimeout(typeNext, profileCode[pos - 1] === '\n' ? 140 : 22); // small pause at each new line
      }
    }
    timer = setTimeout(typeNext, introDelay + 1300);
return () => clearTimeout(timer);
  }, []);
```

```
  // The invisible full text keeps the card at its final height while the visible copy types over it
return (
    <pre aria-hidden="true" className="relative overflow-x-auto whitespace-pre-wrap break-words leading-7
xl:whitespace-pre">
      <code className="invisible">{profileCode}</code>
      <span className="absolute inset-0">
        <code>{profileCode.slice(0, length)}</code><span className="type-cursor">▌</span>
      </span>
    </pre>
  );
}

function Hero() {
  const heroRef = useRef(null);

  // Spotlight follows the mouse (desktop only)   function moveSpotlight(e) {
if (reduceMotion() || !window.matchMedia('(pointer: fine)').matches) return;
const rect = heroRef.current.getBoundingClientRect();
heroRef.current.style.setProperty('--x', `${e.clientX - rect.left}px`);
heroRef.current.style.setProperty('--y', `${e.clientY - rect.top}px`);   }

  // Split the name into letters for the rise animation
let letterIndex = 0;
  const nameWords = 'Ahrone B. Fabro'.split(' ');

  return (
    <section id="home" ref={heroRef} onPointerMove={moveSpotlight} className="relative scroll-mt-20 overflow-hidden">
      <div className="spotlight" aria-hidden="true" />
      <div className="relative mx-auto grid max-w-6xl items-center gap-12 px-5 py-20 md:grid-cols-[1.3fr_1fr] md:py-
28">
        <div>
          <p data-hero="" style={{ '--d': '0s' }} className="font-mono text-sm text-accent dark:text-ember">// hello,
I'm</p>
          <h1 className="mt-3 text-5xl font-bold leading-[1.15] tracking-tight sm:text-6xl lg:text-7xl">
            <span className="sr-only">Ahrone B. Fabro</span>
            <span aria-hidden="true">
              {nameWords.map((word, w) => (
                <span key={word}>
                  <span className="word-mask">
                    {[...word].map((letter) => (
                      <span key={letterIndex} className="char" style={{ '--i': letterIndex++ }}>{letter}</span>
                    ))}
                  </span>
                  {w < nameWords.length - 1 && ' '}
                </span>
              ))}
            </span>
          </h1>
          <p data-hero="" style={{ '--d': '.7s' }} className="mt-6 max-w-xl text-lg text-ink/75 dark:text-chalk/75">IT
student who builds websites and applications that are clean, fast, and easy for everyone to use.</p>
          <div data-hero="" style={{ '--d': '.9s' }} className="mt-8 flex flex-wrap gap-3">
            <a href="#projects" className="group rounded-md bg-accent px-6 py-3 font-medium text-white transition
hover:bg-ink dark:bg-ember dark:text-night dark:hover:bg-chalk">
              View my projects <span aria-hidden="true" className="inline-block transition-transform group-
hover:translate-x-1">→</span>
            </a>
            <a href="#contact" className="rounded-md border border-ink/20 px-6 py-3 font-medium transition
hover:border-ink dark:border-chalk/20 dark:hover:border-chalk">Get in touch</a>
          </div>
        </div>

        <div data-hero="" style={{ '--d': '1.1s' }} className="min-w-0 rounded-xl border border-ink/15 bg-white p-6
font-mono text-sm shadow-[6px_6px_0_0_var(--color-ink)] dark:border-chalk/15 dark:bg-white/5 dark:shadow-
[6px_6px_0_0_var(--color-ember)]">
          <p className="mb-4 flex items-center justify-between border-b border-ink/10 pb-3 text-xs text-ink/70
dark:border-chalk/10 dark:text-chalk/70">
            <span>profile.json</span><span aria-hidden="true">● ● ●</span>
          </p>
          <p className="sr-only">Profile: aspiring web developer studying BS Information Technology, learning Flutter,
 Laravel and Tailwind, open to OJT or internship.</p>
          <TypedCode />
        </div>
      </div>
    </section>
  );
}
```

```
export default function Portfolio({ projects }) {
useScrollReveal([projects]);

  return (
    <>
      <Head title="Portfolio" />

      <div className="intro-curtain" aria-hidden="true">
        <div className="text-center">
          <p className="font-mono text-5xl font-semibold">AF<span className="text-ember">.</span></p>
          <span className="intro-line" />
        </div>
      </div>
      <div className="grain" aria-hidden="true" />
      <ScrollProgress />

      <a href="#main" className="sr-only focus:not-sr-only focus:fixed focus:left-4 focus:top-4 focus:z-50
focus:rounded-md focus:bg-accent focus:px-4 focus:py-2 focus:text-white">Skip to content</a>       <Navbar
links={navLinks} />

      <main id="main">
        <Hero />

        {/* Marquee strip (decorative) */}
        <div className="marquee border-y border-ink/10 bg-ink py-4 text-paper dark:border-chalk/10 dark:bg-chalk
dark:text-night" aria-hidden="true">
          <div className="marquee-track font-mono text-sm uppercase tracking-[0.3em]">
            <span>{marqueeText}</span>
            <span>{marqueeText}</span>
          </div>
        </div>

        {/* About */}
        <section id="about" className="scroll-mt-20 border-b border-ink/10 dark:border-chalk/10">
          <div className="mx-auto grid max-w-6xl gap-10 px-5 py-24 md:grid-cols-[auto_1fr] md:gap-16">
            <div data-reveal="" style={stagger(0)} className="relative size-40 md:size-48">
              <span className="ring-spin absolute -inset-3 rounded-full border-2 border-dashed border-accent
dark:border-ember" aria-hidden="true" />
              <img src="/images/profile.jpg" alt="Photo of Ahrone B. Fabro" width="1024" height="1024" loading="lazy"
decoding="async" className="size-full rounded-full object-cover" />
            </div>
            <div data-reveal="" style={stagger(1)}>
              <SectionTitle number="01" label="about" title="About me" reveal={false} />
              <p className="mt-6 max-w-2xl text-lg text-ink/75 dark:text-chalk/75">I'm Ahrone, a BS Information
Technology student at Urdaneta City University. I started in 2024 after getting curious about how the apps and websites
I used every day were actually put together and I've been building my own ever since. Most of my time goes into front-
end work: HTML, Tailwind CSS, and JavaScript, plus Figma when a project starts as a design.</p>
              <p className="mt-4 max-w-2xl text-lg text-ink/75 dark:text-chalk/75">Right now I'm learning Flutter and
Laravel so I can build a whole project, not just the part people see. I want my work to load fast and stay easy to read
on a phone, not only on a laptop. I'm looking for an OJT or internship where I can build real things with a team and
keep learning.</p>
            </div>
          </div>
        </section>

        {/* Skills */}
        <section id="skills" className="scroll-mt-20 border-b border-ink/10 dark:border-chalk/10">
          <div className="mx-auto max-w-6xl px-5 py-24">
            <SectionTitle number="02" label="skills" title="What I work with" />
            <div className="mt-10 grid gap-6 sm:grid-cols-2 lg:grid-cols-4">
              {skillGroups.map((group, i) => (
                <article key={group.title} data-reveal="" style={stagger(i)} className="rounded-xl border border-
ink/10 bg-white p-6 hover:-translate-y-1.5 hover:shadow-[6px_6px_0_0_var(--color-ink)] dark:border-chalk/10
dark:bg-white/5 dark:hover:shadow-[6px_6px_0_0_var(--color-ember)]">                  <h3 className="text-lg
font-bold">{group.title}</h3>
                  <ul className="mt-4 flex flex-wrap gap-2 font-mono text-sm">
                    {group.items.map((item) => <li key={item} className={skillChip[group.style]}>{item}</li>)}
                  </ul>
                </article>
              ))}
            </div>
          </div>
        </section>

        {/* Projects: the dynamic section, loaded from the database */}
        <ProjectsSection projects={projects} />

        {/* Education */}
```

```
        <section id="education" className="scroll-mt-20 border-b border-ink/10 dark:border-chalk/10">
          <div className="mx-auto max-w-6xl px-5 py-24">
            <SectionTitle number="04" label="education" title="Education" />
            <ol className="mt-10 max-w-2xl space-y-10 border-l-2 border-ink/15 pl-8 dark:border-chalk/15">
              {education.map((item, i) => (
                <li key={item.title} data-reveal="" style={stagger(i)} className="relative">
                  <span className={`absolute -left-[41px] top-1.5 size-4 rounded-full border-2 border-paper
dark:border-night ${item.current ? 'bg-accent dark:bg-ember' : 'bg-ink dark:bg-chalk'}`} aria-hidden="true" />
                  <p className="font-mono text-sm text-accent dark:text-ember">{item.years}</p>
                  <h3 className="mt-1 text-lg font-bold">{item.title}</h3>
                  <p className="text-ink/75 dark:text-chalk/75">{item.school}</p>
                </li>
              ))}
            </ol>
          </div>
        </section>

        {/* Contact */}
        <section id="contact" className="scroll-mt-20">
          <div className="mx-auto grid max-w-6xl gap-12 px-5 py-24 md:grid-cols-2">
            <div data-reveal="" style={stagger(0)}>
              <SectionTitle number="05" label="contact" title="Let's talk" reveal={false} />
              <p className="mt-6 text-lg text-ink/75 dark:text-chalk/75">Open to OJT, internships, and small web
projects. Send a message and I'll get back to you.</p>
              <ul className="mt-8 space-y-3 font-mono text-sm">
                <li><a href="mailto:ahronefabro22@gmail.com" className="underline decoration-accent decoration-2
underline-offset-4 dark:decoration-ember">ahronefabro22@gmail.com</a></li>
              </ul>
            </div>
            <div data-reveal="" style={stagger(1)}>
              <ContactForm />
            </div>
          </div>
        </section>
      </main>

      <Footer />
    </>
  );
}
```

f. Components

`resources/js/Components/ProjectForm.jsx`

This is the reusable form used by both Create and Edit, so the fields are written only once. It reads `errors` from `useForm` and shows each one beside its field, and because `useForm` keeps the data, typed values are not lost when validation fails. After a failed save, it moves to the first invalid field.

```
import { Link } from '@inertiajs/react'; import {
useEffect, useRef, useState } from 'react'; import
FormField, { inputClass } from './FormField'; import
{ categoryLabels } from './ProjectCard';

// The project form used by both Create and Edit.
// `form` is the object from Inertia's useForm(); it keeps the typed values when validation fails.
export default function ProjectForm({ form, categories, onSubmit, submitLabel, currentImageUrl = null }) {
const { data, setData, errors, processing } = form;   const [preview, setPreview] = useState(null);
const formRef = useRef(null);

  // After a failed save, jump to the first problem: the submit button is far below the top fields.
  // Centered because a plain focus() can leave the field hidden behind the sticky header.
  useEffect(() => {     const firstInvalid =
formRef.current.querySelector('[aria-invalid="true"]');     if (!firstInvalid)
return;     firstInvalid.focus({ preventScroll: true });
firstInvalid.scrollIntoView({ block: 'center' });   }, [errors]);

  // Show a preview of a newly picked image   useEffect(() => {
if (!(data.image instanceof File)) return setPreview(null);
const url = URL.createObjectURL(data.image);
setPreview(url);     return () => URL.revokeObjectURL(url);
}, [data.image]);

  function toggleCategory(category) {
setData('categories', data.categories.includes(category)
      ? data.categories.filter((c) => c !== category)
      : [...data.categories, category]);
  }

  // Laravel reports problems with one checkbox as "categories.0", "categories.1", ...
const categoryError = errors.categories     ?? Object.entries(errors).find(([key]) =>
key.startsWith('categories.'))?.[1];

  // Shared props that connect each input to its error message
const a11y = (field) => ({     id: field,
    'aria-invalid': errors[field] ? 'true' : 'false',
    'aria-describedby': `${field}-error`,
  });

  function handleSubmit(e) {
    e.preventDefault();
onSubmit();   }

  return (
    // noValidate: let Laravel do the checking so its error messages are the ones shown
    <form ref={formRef} onSubmit={handleSubmit} noValidate className="space-y-5 rounded-xl border border-ink/10 bg-
white p-6 dark:border-chalk/10 dark:bg-white/5">
      <FormField id="title" label="Title" error={errors.title}>
        <input {...a11y('title')} type="text" value={data.title} onChange={(e) => setData('title', e.target.value)}
className={inputClass} />       </FormField>

      <fieldset aria-describedby="categories-error">
        <legend className="font-medium">Categories</legend>
        <div className="mt-2 flex flex-wrap gap-2">
          {categories.map((category) => (
            <label key={category} className="flex items-center gap-2 rounded-full border border-ink/20 px-4 py-2
text-sm has-checked:border-ink has-checked:bg-ink has-checked:text-paper dark:border-chalk/20 dark:has-
checked:border-chalk dark:has-checked:bg-chalk dark:has-checked:text-night">               <input
type="checkbox" aria-invalid={categoryError ? 'true' : 'false'}
checked={data.categories.includes(category)} onChange={() => toggleCategory(category)} className="accent-accent" />
```

```
              {categoryLabels[category] ?? category}
            </label>
          ))}
        </div>
        <p id="categories-error" className="mt-1 text-sm text-red-700 dark:text-red-400">{categoryError}</p>
      </fieldset>

      <FormField id="description" label="Description" error={errors.description} hint={`${data.description.length} /
300 characters`}>
        <textarea {...a11y('description')} rows={4} value={data.description} onChange={(e) => setData('description',
e.target.value)} className={inputClass} />       </FormField>

      <FormField id="tech_stack" label="Tools used" error={errors.tech_stack} hint="Separate with commas, e.g. Figma,
Flutter">
        <input {...a11y('tech_stack')} type="text" value={data.tech_stack} onChange={(e) => setData('tech_stack',
e.target.value)} className={inputClass} />       </FormField>

      <FormField id="project_url" label="Project link" optional error={errors.project_url}>
        <input {...a11y('project_url')} type="url" placeholder="https://" value={data.project_url ?? ''} onChange={(e)
 => setData('project_url', e.target.value)} className={inputClass} />
      </FormField>

      <FormField id="image" label="Cover image" optional={Boolean(currentImageUrl)} error={errors.image}
        hint={currentImageUrl ? 'JPG, PNG or WEBP, up to 2 MB. Leave empty to keep the current image.' : 'JPG, PNG or
WEBP, up to 2 MB.'}>
        <input {...a11y('image')} type="file" accept="image/jpeg,image/png,image/webp" onChange={(e) =>
setData('image', e.target.files[0] ?? null)}           className={`${inputClass} file:mr-4 file:rounded
file:border-0 file:bg-ink file:px-3 file:py-1 file:text-paper dark:file:bg-chalk dark:file:text-night`}
/>
        {(preview || currentImageUrl) && (
          <img src={preview ?? currentImageUrl} alt={preview ? 'Preview of the new image' : 'Current image'}
className="mt-3 h-32 rounded-md border border-ink/10 object-cover dark:border-chalk/10" />
        )}
      </FormField>

      <FormField id="image_alt" label="Image description (alt text)" error={errors.image_alt} hint="What the image
shows, for visitors using a screen reader.">
        <input {...a11y('image_alt')} type="text" value={data.image_alt} onChange={(e) => setData('image_alt',
e.target.value)} className={inputClass} />       </FormField>

      <div className="grid gap-5 sm:grid-cols-2">
        <FormField id="date_completed" label="Date completed" error={errors.date_completed}>
          <input {...a11y('date_completed')} type="date" value={data.date_completed} onChange={(e) =>
setData('date_completed', e.target.value)} className={inputClass} />
        </FormField>
        <FormField id="display_order" label="Display order" error={errors.display_order} hint="1 shows first.">
          <input {...a11y('display_order')} type="number" value={data.display_order} onChange={(e) =>
setData('display_order', e.target.value)} className={inputClass} />
        </FormField>
      </div>

      <div className="flex flex-wrap items-center gap-3 pt-2">
        <button type="submit" disabled={processing} className="rounded-md bg-accent px-6 py-3 font-medium text-white
transition hover:bg-ink disabled:opacity-60 dark:bg-ember dark:text-night dark:hover:bg-chalk">
          {processing ? 'Saving…' : submitLabel}
        </button>
        <Link href="/projects" className="rounded-md border border-ink/20 px-6 py-3 font-medium hover:border-ink
dark:border-chalk/20 dark:hover:border-chalk">Cancel</Link>
      </div>
    </form>
  );
}
```

resources/js/Components/FormField.jsx

This small component draws one field: its label, the input passed in as children, an optional hint and the error message. Every field in the form therefore looks and behaves the same.

```
export const inputClass =
  'mt-2 w-full rounded-md border border-ink/20 bg-paper px-4 py-3 aria-[invalid=true]:border-red-700 dark:border-
chalk/20 dark:bg-night dark:aria-[invalid=true]:border-red-400';

// A label, the input passed in as children, an optional hint, and the error message under it.
// The input should use id={id}, aria-invalid={!!error} and aria-describedby={`${id}-error`}.
export default function FormField({ id, label, error, hint, optional = false, children }) {
return (     <div>
      <label htmlFor={id} className="font-medium">
        {label} {optional && <span className="font-normal text-ink/60 dark:text-chalk/60">(optional)</span>}
      </label>
      {children}
      {hint && <p className="mt-1 text-sm text-ink/60 dark:text-chalk/60">{hint}</p>}
      <p id={`${id}-error`} className="mt-1 text-sm text-red-700 dark:text-red-400">{error}</p>
    </div>
  );
}
```

resources/js/Components/ConfirmDialog.jsx

This is the pop-up that asks for confirmation before deleting. It uses the browser's `dialog` `<>` element, so Escape and keyboard focus already work.

```
import { useEffect, useRef } from 'react';

// "Are you sure?" pop-up shown before a record is deleted.
// Uses the native <dialog> element, so Escape and focus trapping work for free.
export default function ConfirmDialog({ open, title, message, confirmLabel = 'Delete', processing = false, onConfirm,
onCancel }) {
  const dialogRef = useRef(null);

  useEffect(() => {     const dialog =
dialogRef.current;     if (open && !dialog.open)
dialog.showModal();     if (!open && dialog.open)
dialog.close();   }, [open]);

  return (     <dialog       ref={dialogRef}       onCancel={(e) => { e.preventDefault(); onCancel(); }}
aria-labelledby="confirm-title"       className="m-auto w-[calc(100%-2rem)] max-w-md rounded-xl border
border-ink/15 bg-paper p-6 text-ink shadow-[6px_6px_0_0_var(--color-ink)] backdrop:bg-ink/60 dark:border-
chalk/15 dark:bg-night dark:text-chalk dark:shadow-[6px_6px_0_0_var(--color-ember)]"
    >
      <h2 id="confirm-title" className="text-xl font-bold">{title}</h2>
      <p className="mt-3 text-ink/75 dark:text-chalk/75">{message}</p>
      <div className="mt-6 flex justify-end gap-3">
        <button type="button" onClick={onCancel} autoFocus className="rounded-md border border-ink/20 px-4 py-2 font-
medium hover:border-ink dark:border-chalk/20 dark:hover:border-chalk">
          Cancel
        </button>
        <button type="button" onClick={onConfirm} disabled={processing} className="rounded-md bg-red-700 px-4 py-2
font-medium text-white hover:bg-red-800 disabled:opacity-60 dark:bg-red-500 dark:text-night dark:hover:bg-red-400">
          {processing ? 'Deleting…' : confirmLabel}
        </button>
      </div>
    </dialog>
  );
}
```

resources/js/Components/FlashMessage.jsx

This component shows the success message that the controller sets with `- with('success', ...)>`. It appears after create, update and delete, and the user can dismiss it.

```
import { usePage } from '@inertiajs/react';
import { useEffect, useState } from 'react';

// Shows the controller's ->with('success', ...) message after create, update and delete.
export default function FlashMessage() {   const { flash } = usePage().props;
  const [visible, setVisible] = useState(Boolean(flash.success));

  // Show it again whenever a new message arrives
  useEffect(() => { setVisible(Boolean(flash.success)); }, [flash]);

if (!visible) return null;

  return (
    <div role="status" className="mb-6 flex items-start justify-between gap-4 rounded-md border border-green-700 bg-
green-50 p-4 text-green-800 dark:border-green-400 dark:bg-green-950 dark:text-green-200">
      <p>{flash.success}</p>
      <button type="button" onClick={() => setVisible(false)} aria-label="Dismiss message" className="font-
mono">×</button>
    </div>
  );
}
```

resources/js/Components/ProjectsSection.jsx

This is the Projects section on the public page. It shows the filter buttons and keeps only the projects that match the selected category.

```
import { useState } from 'react'; import
ProjectCard from './ProjectCard'; import
SectionTitle from './SectionTitle';

const filters = ['all', 'web', 'mobile', 'design', 'school']; const filterBtnClass = 'rounded-
full border border-ink/20 px-4 py-2 text-sm capitalize transition aria-pressed:border-ink aria-
pressed:bg-ink aria-pressed:text-paper dark:border-chalk/20 dark:aria-pressed:border-chalk
dark:aria-pressed:bg-chalk dark:aria-pressed:text-night';

// "Things I've built": the filter buttons and the project cards from the database. export default
function ProjectsSection({ projects }) {   const [filter, setFilter] = useState('all');   const
[hasFiltered, setHasFiltered] = useState(false);   const visible = filter === 'all' ? projects :
projects.filter((p) => p.categories.includes(filter));

  function choose(value)
{     setFilter(value);
setHasFiltered(true);   }

  const status = visible.length === 0
    ? (filter === 'all' ? 'No projects yet.' : `No ${filter} projects yet.`)
    : `Showing ${visible.length} ${filter === 'all' ? '' : filter + ' '}project${visible.length === 1 ? '' : 's'}.`;

  return (
    <section id="projects" className="scroll-mt-20 border-b border-ink/10 dark:border-chalk/10">
      <div className="mx-auto max-w-6xl px-5 py-24">
        <SectionTitle number="03" label="projects" title="Things I've built" />

        <div  data-reveal=""  className="mt-8  flex  flex-wrap  gap-2"  role="group"  aria-label="Filter  projects  by
category">
          {filters.map((value) => (
            <button key={value} type="button" onClick={() => choose(value)} aria-pressed={filter === value}
className={filterBtnClass}>
              {value}
            </button>
          ))}
        </div>
        <p aria-live="polite" className="mt-4 font-mono text-sm text-ink/70 dark:text-chalk/70">{status}</p>

        <div className="mt-8 grid gap-6 sm:grid-cols-2 lg:grid-cols-3">
          {visible.map((project, i) => (
            // A new key after each filter click re-mounts the card so its entrance animation plays again
            <ProjectCard
              key={hasFiltered ? `${filter}-${project.id}` :
project.id}               project={project}
reveal={!hasFiltered}               delay={i * 120}
            />
          ))}
        </div>
      </div>
    </section>
  );
}
```

resources/js/Components/ProjectCard.jsx

This component shows one project on the public page. The same card is used for every record, and the link only appears if the project has one.

```
export const categoryLabels = { web: 'Web', mobile: 'Mobile app', design: 'Design', school: 'School' };

const formatDate = (value) =>
  new Date(`${value}T00:00:00`).toLocaleDateString('en-US', { month: 'short', year: 'numeric' });

// One project on the public portfolio. `reveal` = fade in on scroll; otherwise it animates in after a filter click.
export default function ProjectCard({ project, reveal, delay = 0 }) {
  const tools = project.tech_stack.split(',').map((tool) => tool.trim()).filter(Boolean);

  return (
<article
      data-reveal={reveal ? '' : undefined}       style={{ '--d': `${delay}ms` }}       className={`group flex flex-
col overflow-hidden rounded-xl border border-ink/10 bg-white hover:-translate-y-1.5 hover:shadow-[6px_6px_0_0_var(--
color-ink)] dark:border-chalk/10 dark:bg-white/5 dark:hover:shadow-[6px_6px_0_0_var(--color-ember)] ${reveal ? '' :
'filter-in'}`}
    >
      <div className="relative flex h-48 items-end overflow-hidden bg-ink p-5 dark:bg-chalk/10">
        <img
          src={project.image_url}           alt={project.image_alt}           loading="lazy"
decoding="async"           className="absolute inset-0 size-full object-cover object-top transition-
transform duration-500 group-hover:scale-105"
        />
        <span className="absolute inset-0 bg-linear-to-t from-ink/85 via-ink/20 to-transparent" aria-hidden="true" />
        <span className="relative font-mono text-5xl font-semibold text-paper transition-transform duration-500 group-
hover:translate-x-3" aria-hidden="true">
          {String(project.display_order).padStart(2, '0')}
        </span>
      </div>

      <div className="flex flex-1 flex-col p-6">
        <p className="font-mono text-xs uppercase tracking-wider text-accent dark:text-ember">
          {project.categories.map((c) => categoryLabels[c] ?? c).join(' · ')}
        </p>
        <h3 className="mt-2 text-xl font-bold">{project.title}</h3>
        <p className="mt-3 flex-1 text-ink/75 dark:text-chalk/75">{project.description}</p>

        <ul className="mt-4 flex flex-wrap gap-2 font-mono text-xs" aria-label="Tools used">
          {tools.map((tool) => (
            <li key={tool} className="rounded border border-ink/20 px-2 py-1 dark:border-chalk/20">{tool}</li>
          ))}
        </ul>

        <div className="mt-4 flex items-center justify-between font-mono text-xs text-ink/70 dark:text-chalk/70">
          <span>Completed {formatDate(project.date_completed)}</span>
          {project.project_url && (
            <a href={project.project_url} target="_blank" rel="noopener noreferrer" className="text-accent underline
underline-offset-4 hover:no-underline dark:text-ember">
              View project<span className="sr-only"> {project.title} (opens in a new tab)</span> ↗
            </a>
          )}
        </div>
      </div>
    </article>
  );
}
```

resources/js/Components/Navbar.jsx

This is the top bar of the public page. It has the section links, the theme toggle and a mobile menu, and it highlights the section being viewed.

```
import { useEffect, useState } from 'react';
import ThemeToggle from './ThemeToggle';

const linkClass = 'hover:text-accent aria-[current=true]:font-semibold aria-[current=true]:text-accent
dark:hover:text-ember dark:aria-[current=true]:text-ember';

// Sticky top bar: logo, section links (highlighted while you scroll), theme toggle, mobile menu.
export default function Navbar({ links }) {   const [menuOpen, setMenuOpen] = useState(false);
const [active, setActive] = useState(null);

  // Close the mobile menu with Escape   useEffect(() => {     const
onKey = (e) => { if (e.key === 'Escape') setMenuOpen(false); };
document.addEventListener('keydown', onKey);     return () =>
document.removeEventListener('keydown', onKey);   }, []);

  // Highlight the link of the section in the middle of the screen   useEffect(() => {
const observer = new IntersectionObserver((entries) => {       entries.forEach((entry) => {
if (entry.isIntersecting) setActive(entry.target.id); });
    }, { rootMargin: '-45% 0px -50% 0px' });
    document.querySelectorAll('main section[id]').forEach((section) => observer.observe(section));
return () => observer.disconnect();   }, []);

  return (
    <header className="sticky top-0 z-40 border-b border-ink/10 bg-paper/90 backdrop-blur dark:border-chalk/10 dark:bg-
night/90">
      <nav className="mx-auto flex max-w-6xl items-center justify-between px-5 py-4" aria-label="Main">
        <a href="#home" className="font-mono text-sm font-semibold">My<span className="text-accent dark:text-ember">
Portfolio</span></a>

        <ul className="hidden gap-8 text-sm md:flex">
          {links.map((link) => (
            <li key={link.id}>
              <a href={`#${link.id}`} aria-current={active === link.id ? 'true' : undefined}
className={linkClass}>{link.label}</a>
            </li>
          ))}
        </ul>

        <div className="flex items-center gap-2">
          <ThemeToggle />
          <button             type="button"             onClick={() => setMenuOpen(!menuOpen)}
aria-controls="mobile-menu"             aria-expanded={menuOpen}             aria-label={menuOpen ?
'Close menu' : 'Open menu'}             className="grid size-10 place-items-center rounded-md border
border-ink/15 text-lg md:hidden dark:border-chalk/15"
          >
            {menuOpen ? '✕' : '☰'}
          </button>
        </div>
      </nav>

      <div id="mobile-menu" hidden={!menuOpen} className="border-t border-ink/10 md:hidden dark:border-chalk/10">
        <ul className="flex flex-col px-5 py-2 text-base">
          {links.map((link) => (
            <li key={link.id}>
              <a href={`#${link.id}`} onClick={() => setMenuOpen(false)} className="block py-3">{link.label}</a>
            </li>
          ))}
        </ul>
      </div>
    </header>
  );
}
```

resources/js/Components/Footer.jsx

This is the bottom of the public page. It shows the copyright line and the fonts used on the site.

```
export default function Footer() {
return (
    <footer className="border-t border-ink/10 dark:border-chalk/10">
      <div className="mx-auto flex max-w-6xl flex-col gap-2 px-5 py-8 font-mono text-xs text-ink/70 sm:flex-row
sm:justify-between dark:text-chalk/70">
        <p>© 2026 Ahrone B. Fabro · Built with Laravel, Inertia, React &amp;amp; Tailwind CSS</p>
        <p>Fonts: Space Grotesk &amp;amp; JetBrains Mono (Google Fonts)</p>
      </div>
    </footer>
  );
}
```

resources/js/Components/SectionTitle.jsx

This component draws the small label and big heading at the top of each section, so every section title looks the same.

```
// The "03 / projects" label + big heading used at the top of every section.
export default function SectionTitle({ number, label, title, reveal = true }) {
return (
    <div data-reveal={reveal ? '' : undefined}>
      <p className="font-mono text-sm text-accent dark:text-ember">{number} / {label}</p>
<h2 className="mt-2 text-3xl font-bold tracking-tight sm:text-4xl">{title}</h2>
    </div>
  );
}
```

resources/js/Components/ThemeToggle.jsx

This is the light and dark mode button used on both the public and management pages. It saves the choice in the browser and follows the device setting if no choice was made.

```
import { useEffect, useRef, useState } from 'react'; import { flushSync } from 'react-
dom'; const reduceMotion = () => window.matchMedia('(prefers-reduced-motion:
reduce)').matches;

function saveTheme(value) {   try {
localStorage.setItem('theme', value);
  } catch (err) {
    // Storage blocked: the theme still switches, it just won't be remembered
  }
}

// Dark / light switch. The starting theme is set in app.blade.php before the page paints.
export default function ThemeToggle() {   const [isDark, setIsDark] = useState(() =>
document.documentElement.classList.contains('dark'));   const btnRef = useRef(null);

  function applyTheme(dark) {
document.documentElement.classList.toggle('dark', dark);
setIsDark(dark);   }

  function toggle() {
const goDark = !isDark;
    saveTheme(goDark ? 'dark' : 'light');

    // Instant switch if the browser has no View Transitions or the user prefers less motion
if (!document.startViewTransition || reduceMotion()) {       applyTheme(goDark);
return;     }

    // Circle reveal that grows out of the toggle button
const rect = btnRef.current.getBoundingClientRect();
const x = rect.left + rect.width / 2;     const y =
rect.top + rect.height / 2;
    const radius = Math.hypot(Math.max(x, window.innerWidth - x), Math.max(y, window.innerHeight - y));

    const transition = document.startViewTransition(() => flushSync(() => applyTheme(goDark)));
transition.ready.then(() => {       document.documentElement.animate(
        { clipPath: [`circle(0px at ${x}px ${y}px)`, `circle(${radius}px at ${x}px ${y}px)`] },
        { duration: 650, easing: 'cubic-bezier(.65, 0, .35, 1)', pseudoElement: '::view-transition-new(root)' },
      );
    }).catch(() => {});
  }

  // If the visitor never picked a theme, follow their device setting live
useEffect(() => {     const media = window.matchMedia('(prefers-color-
scheme: dark)');     function onChange(event) {       let saved = null;
try { saved = localStorage.getItem('theme'); } catch (err) {}       if
(!saved) applyTheme(event.matches);
    }
    media.addEventListener('change', onChange);     return ()
=> media.removeEventListener('change', onChange);   }, []);

  return (     <button       ref={btnRef}       type="button"       onClick={toggle}       aria-
label={isDark ? 'Switch to light mode' : 'Switch to dark mode'}       className="grid size-10 place-
items-center rounded-md border border-ink/15 text-lg hover:border-ink dark:border-chalk/15
dark:hover:border-chalk"
    >
      {isDark ? '☀' : '☾'}
    </button>
  );
}
```

resources/js/Components/ScrollProgress.jsx

This is the thin bar at the top of the public page that fills as the visitor scrolls down.

```
import { useEffect, useRef } from 'react';

// Thin bar across the top that fills as you scroll down the page.
export default function ScrollProgress() {   const barRef =
useRef(null);

  useEffect(() => {     function update() {       const maxScroll =
document.documentElement.scrollHeight - window.innerHeight;       const progress =
maxScroll > 0 ? window.scrollY / maxScroll : 0;
barRef.current.style.transform = `scaleX(${progress})`;
    }
    window.addEventListener('scroll', update, { passive: true });
update();     return () => window.removeEventListener('scroll',
update);   }, []);

  return (     <div       ref={barRef}       className="fixed left-0 top-0 z-50 h-1 w-
full origin-left bg-accent dark:bg-ember"       style={{ transform: 'scaleX(0)' }}
aria-hidden="true"
    />
  );
}
```

resources/js/Components/ContactForm.jsx

This form checks the fields in the browser and autosaves a draft in localStorage. It has no back-end, so it does not send or store the message.

```
import { useEffect, useRef, useState } from 'react';
import FormField, { inputClass } from './FormField';

// Each dot-separated part must be non-empty, so "a@b..com" fails
const emailPattern = /^[^\s@.]+(\.[^\s@.]+)*@[^\s@.]+(\.[^\s@.]+)*\.[^\s@.]{2,}$/;
const empty = { name: '', email: '', message: '' };

function readDraft() {   try {     const draft = JSON.parse(localStorage.getItem('contact-draft')) || {};
return Object.fromEntries(Object.keys(empty).map((k) => [k, typeof draft[k] === 'string' ? draft[k] : '']));
  } catch (err) {
    return empty; // No draft, or one that can't be read: start with an empty form
  }
}

// Front-end-only contact form, plus my unique feature: the draft is autosaved to localStorage
// so a refresh or an accidental tab close doesn't wipe a half-written message.
export default function ContactForm() {   const [values, setValues] =
useState(empty);   const [errors, setErrors] = useState({});   const [success,
setSuccess] = useState('');   const [draftRestored, setDraftRestored] =
useState(false);
  const refs = { name: useRef(null), email: useRef(null), message: useRef(null) };

  // On load, put the saved draft back into the fields
useEffect(() => {     const draft = readDraft();
if (Object.values(draft).some(Boolean)) {
setValues(draft);       setDraftRestored(true);
    }
  }, []);

  function saveDraft(next) {     try {
localStorage.setItem('contact-draft', JSON.stringify(next));
    } catch (err) {
      // Storage blocked: the form still works, the draft just won't be remembered
    }
  }

  function removeDraft() {     try {
localStorage.removeItem('contact-draft'); } catch (err) {}
setDraftRestored(false);   }

  function change(field, value) {     const
next = { ...values, [field]: value };
setValues(next);     saveDraft(next);   }

  function clearDraft() {
setValues(empty);
removeDraft();
refs.name.current.focus();   }

  function handleSubmit(e) {
    e.preventDefault();
setSuccess('');

    const name = values.name.trim();
const email = values.email.trim();
const message = values.message.trim();

    const found = {       name: name ? '' :
'Please enter your name.',       email: !email ?
'Please enter your email.'
        : emailPattern.test(email) ? '' : 'Please enter a valid email, like name@example.com.',
message: message.length >= 10 ? '' : 'Your message should be at least 10 characters.',
    };
```

```
    setErrors(found);

    const firstInvalid = Object.keys(found).find((field) => found[field]);
if (firstInvalid) {       refs[firstInvalid].current.focus();
return;     }

    setSuccess(`Thanks, ${name}! Your message was submitted.`);
setValues(empty);     removeDraft();   }

  const a11y = (field) => ({
id: field,     ref:
refs[field],
    'aria-invalid': errors[field] ? 'true' : 'false',
    'aria-describedby': `${field}-error`,
  });

  return (
    <form onSubmit={handleSubmit} noValidate className="space-y-5 rounded-xl border border-ink/10 bg-white p-6
dark:border-chalk/10 dark:bg-white/5">
      {draftRestored && (
        <p role="status" className="font-mono text-sm text-ink/70 dark:text-chalk/70">
          Draft restored from your last visit.{' '}
          <button type="button" onClick={clearDraft} className="text-accent underline underline-offset-4 hover:no-
underline dark:text-ember">Clear draft</button>
        </p>
      )}
      <FormField id="name" label="Name" error={errors.name}>
        <input {...a11y('name')} name="name" type="text" autoComplete="name" required value={values.name} onChange={(e)
=> change('name', e.target.value)} className={inputClass} />
      </FormField>
      <FormField id="email" label="Email" error={errors.email}>
        <input {...a11y('email')} name="email" type="email" autoComplete="email" required value={values.email}
onChange={(e) => change('email', e.target.value)} className={inputClass} />
      </FormField>
      <FormField id="message" label="Message" error={errors.message}>
        <textarea {...a11y('message')} name="message" rows={5} required value={values.message} onChange={(e) =>
change('message', e.target.value)} className={inputClass} />
      </FormField>
      <button type="submit" className="w-full rounded-md bg-accent px-6 py-3 font-medium text-white transition
hover:bg-ink dark:bg-ember dark:text-night dark:hover:bg-chalk">Send message</button>
      {success && (
        <p role="status" className="rounded-md border border-green-700 bg-green-50 p-4 text-green-800 dark:border-
green-400 dark:bg-green-950 dark:text-green-200">{success}</p>
      )}
    </form>
  );
}
```

resources/js/Layouts/AdminLayout.jsx

This layout is the frame around the Index, Create and Edit pages: the top bar with the Log out button, the page heading and the success message. Because it lives here, all three pages share it without repeating the code.

```
import { Head, Link } from '@inertiajs/react'; import
FlashMessage from '../Components/FlashMessage'; import
ThemeToggle from '../Components/ThemeToggle';

// Frame around the project management pages: top bar, page heading, success message.
export default function AdminLayout({ title, actions = null, children }) {
return (
    <>
      <Head title={title} />
      <a href="#main" className="sr-only focus:not-sr-only focus:fixed focus:left-4 focus:top-4 focus:z-50
focus:rounded-md focus:bg-accent focus:px-4 focus:py-2 focus:text-white">Skip to content</a>
      <header  className="sticky  top-0  z-40  border-b  border-ink/10  bg-paper/90  backdrop-blur  dark:border-chalk/10
dark:bg-night/90">
        <nav className="mx-auto flex max-w-5xl items-center justify-between px-5 py-4" aria-label="Main">
          <Link href="/" className="font-mono text-sm font-semibold">My<span className="text-accent dark:text-ember">
Portfolio</span></Link>
          <div className="flex items-center gap-4 text-sm">
            <Link href="/projects" className="hover:text-accent dark:hover:text-ember">Projects</Link>
            <Link href="/#projects" className="hover:text-accent dark:hover:text-ember">View site</Link>
            <Link  href="/logout"  method="post"  as="button"  className="hover:text-accent  dark:hover:text-ember">Log
out</Link>
            <ThemeToggle />
          </div>
        </nav>
      </header>

      <main id="main" className="mx-auto max-w-5xl px-5 py-12">
        <div className="mb-8 flex flex-wrap items-end justify-between gap-4">
          <div>
            <p className="font-mono text-sm text-accent dark:text-ember">// manage</p>
            <h1 className="mt-2 text-3xl font-bold tracking-tight sm:text-4xl">{title}</h1>
          </div>
          {actions}
        </div>
        <FlashMessage />
        {children}
      </main>
    </>
  );
}
```

app/Http/Middleware/HandleInertiaRequests.php

I added the `flash` value here so that the one-time success message from the session reaches every React page as a prop.

```php
<?php namespace

App\Http\Middleware;

use Illuminate\Http\Request;
use Inertia\Middleware;

class HandleInertiaRequests extends Middleware
{
    /**      * The root template that's loaded on the first
page visit.
     *
* @see https://inertiajs.com/server-side-setup#root-template
     *
* @var string
     */
    protected $rootView = 'app';

    /**      * Determines the current asset
version.
     *
* @see https://inertiajs.com/asset-versioning
     */
    public function version(Request $request): ?string
    {
        return parent::version($request);
}

    /**      * Define the props that are shared by
default.
     *
* @see https://inertiajs.com/shared-data
     *
* @return array<string, mixed>
     */
    public function share(Request $request): array
    {
        return [
            ...parent::share($request),
            // One-time success message set by the controller with ->with('success', ...)
            'flash' => [
                'success' => fn () => $request->session()->get('success'),
            ],
        ];
    }
}
```

5. Design Decisions

---

a. Why did you use a separate resource controller?

Writing everything in `web.php` would mix routes with logic and make that file very long. With `ProjectController`, all the code for projects is in one place and each action has a clear job. It also follows the standard names that `Route::resource` expects, so one route line connects all the actions.

b. Why did you choose your validation rules? Where did you place them and why?

Each rule protects something that would break or look wrong on the public page. For example, a very long title would break the card layout, an invalid link would lead visitors nowhere, and a very large image would slow the page down. I placed the rules in the controller, in a private `rules()` method, because the server is the only place the user cannot bypass. Browser checks can be turned off, so the form uses `noValidate` and shows Laravel's messages. Sharing one method means `store` and `update` cannot drift apart.

c. How did you organize your React pages and components, and why?

Pages that have their own URL are in `Pages/Projects/`, because Inertia finds them by that name. Parts that are used more than once or have a single job are in `Components/`, like `ProjectForm` and `ConfirmDialog`. The shared frame of the management pages is in `Layouts/`. This way, a change to a form field only needs to be made in one file, and it applies to both Create and Edit.

d. Why are the project pages behind a login?

Without a login, any visitor could add, edit or delete projects and upload files. The project routes are now in an `auth` group, so only a logged-in user can reach them. There is no public sign-up page, and the public portfolio stays open to everyone.

6. Validation Test Table

These tests were run through the actual form in Google Chrome on November 2, 2026, using a temporary copy of the database. Each step was driven by an automated browser script. The "Actual Result" column records what the page showed.

#	Test Case	Input Used	Expected Result	Actual Result	Pass / Fail
1	Empty form	All fields blank	Error message under each required field	8 errors appeared, one under each required field. The optional project link had no error. Nothing was saved. (Figure G)	✓ Pass
2	Exceeding maximum length	Title of 61 characters	Error under Title	"The title field must not be greater than 60 characters." appeared under the field. Nothing was saved and the other typed values stayed in the form.	✓ Pass
3	Invalid URL	Project link: `not a link`	Custom error under Project link	"The project link must be a full web address starting with http:// or https://." appeared under the field. Nothing was saved and the other typed values stayed in the form.	✓ Pass
4	Letters in a number field	Display order: `abc` (see note below)	Error under Display order	"The display order field must be an integer." appeared under the field. Nothing was saved and the other typed values stayed in the form.	✓ Pass
5	Number out of range	Display order: `150`	Custom error under Display order	"The display order must be a number from 1 to 99." appeared under the field. Nothing was saved and the other typed values stayed in the form.	✓ Pass
6	Future date	Date completed: 05/01/2030	Custom error under Date completed	"The completion date cannot be in the future." appeared under the field. Nothing was saved and the other typed values stayed in the form.	✓ Pass
7	Duplicate unique value	Title of an existing project	Custom error under Title	"You already have a project with this title." appeared under the field. Nothing was saved and the other typed values stayed in the form.	✓ Pass
#	**Test Case**	**Input Used**	**Expected Result**	**Actual Result**	**Pass / Fail**

8	Updating without changing the unique field	Edited only the description and kept the same title	Saves with a success message and no title error	"Project "Documentation Test Project" was updated." appeared on the Index page.	✓ Pass
9	File that is not an image	Uploaded `notes.pdf`	Custom error under Cover image	"The file must be an image." appeared under the field. Nothing was saved and the other typed values stayed in the form.	✓ Pass
10	Image that is too large	Uploaded a JPG of about 3 MB	Custom error under Cover image	"The image must be 2 MB or smaller." appeared under the field. Nothing was saved and the other typed values stayed in the form.	✓ Pass
11	Valid data	All fields filled correctly	Record saved, success message shown, record appears on the public page	"Project "Documentation Test Project" was added." appeared, and the project was shown on the public portfolio page. (Figure F)	✓ Pass
12	Delete after confirming	Clicked Delete, then confirmed in the dialog	Record removed, success message shown	"Project "Documentation Test Project" was deleted." appeared, and the project was no longer in the list. (Figure E)	✓ Pass

Note on test 4: Chrome does not let you type letters into a number input. To prove that the server still rejects them, the input's type was changed to text for this test and `abc` was then submitted.

Failed tests: No test failed in this run, so no fixes were needed.

Automated tests: `tests/Feature/ProjectCrudTest.php` covers the same cases, and `tests/Feature/LoginTest.php` covers the login. Running `php artisan test` gave 21 passed tests with 108 assertions.

7. Reflection

---

a. What was the hardest part of converting your portfolio, and how did you solve it?

The hardest part was making image uploads work on the Edit page. The form sends a file, but browsers cannot send files with a real PUT request, so my update route would not receive the data correctly. I solved it by sending a POST request with _method set to put, which Laravel treats as a PUT. I also had to make sure the old image stays when no new file is chosen, and that it is deleted from storage when it is replaced.

b. What did you learn about validation that you did not know before?

I learned that validation has to happen on the server, because anything in the browser can be bypassed. I saw this myself when I changed a number input to a text input and Laravel still rejected the letters. I also learned that a unique rule needs to ignore the current record on update, or a project could not be saved with its own title, and that Inertia delivers the errors to React without a full page reload, so the typed values stay in the form.

c. If you had to add a second dynamic section, what would you do differently?

I would reuse FormField, ConfirmDialog, FlashMessage and AdminLayout, since they do not depend on projects. I would also move the validation rules out of the controller into a Form Request class, so each section has its rules in its own file and the controllers stay short. Finally, I would write the automated tests earlier, so I could catch mistakes as I build each part.

8. Links

Item	Link
GitHub repository	https://github.com/Ahronefabro/Portfolio_Laravel.git
Video demonstration	[Paste your Google Drive or unlisted YouTube link]